

Лабораторная работа № 5

Моделирование с использованием сетей Петри

Нечаева Кира Андреевна

НКНбд-01-23

2026-01-01

Цель работы

- Реализовать задачу «Обедающие философы» с помощью сети Петри.
- Исследовать возникновение взаимной блокировки.
- Сравнить классическую сеть и сеть с арбитром.
- Выполнить стохастическое и детерминированное моделирование.
- Исследовать влияние параметров модели.
- Проверить воспроизводимость вычислений.

У каждого философа есть
состояния:

- Think;
- Hungry;
- Eat.

Вилки представлены
отдельными позициями сети.
Переходы соответствуют
захвату и освобождению
ресурсов.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/test_petri_net.jl
Создание классической сети...
Позиции: 20
Переходы: 15
Начальное число фишек: 10.0

Создание сети с арбитром...
Позиции: 21
Переходы: 15
Фишек арбитра: 4.0

Тестовая стохастическая симуляция...
Получено состояний: 6

Ядро сети Петри работает.
```

Рисунок 1: Проверка реализации сети

Почему возникает deadlock

В классической постановке каждый философ сначала может получить одну вилку.

Если все философы одновременно удерживают по одной вилке:

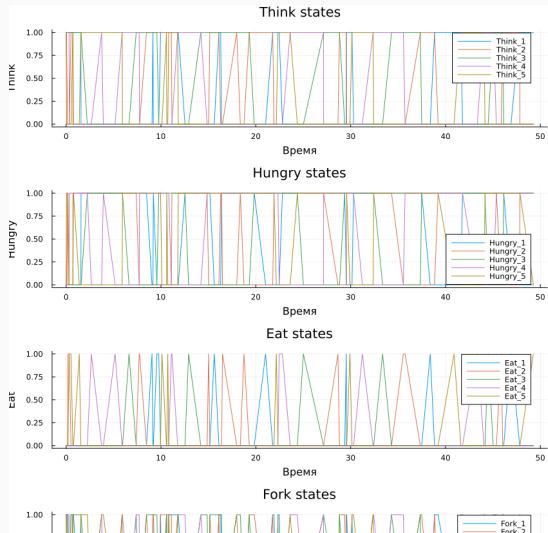
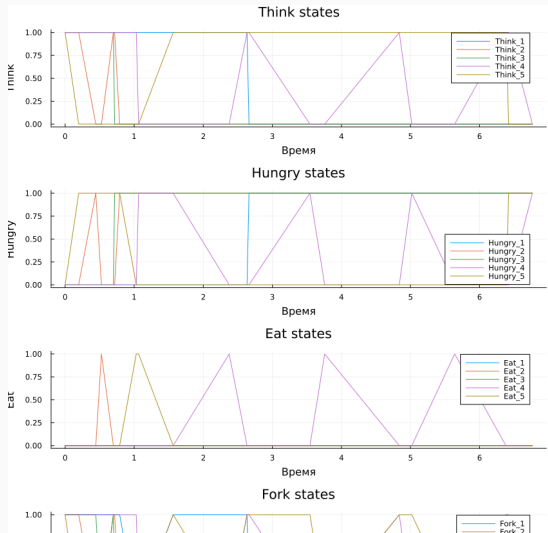
- соседняя вилка уже занята;
- перейти в состояние Eat невозможно;
- ни один необходимый переход не может сработать;
- сеть оказывается во взаимной блокировке.

В модифицированную сеть добавлена дополнительная позиция *Arbiter*.

В ней находится $N - 1$ фишек.

Это ограничивает число философов, которые могут одновременно начать захват ресурсов, и разрушает условие возникновения общего тупика.

Стохастическая модель



Программная проверка deadlock

После симуляции программа проверяет последнюю маркировку.

Если ни один переход больше не разрешён, состояние определяется как `deadlock`.

Для сравнения две модели запускаются с одинаковым `seed`.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers.jl
N = 5
tmax = 50.0

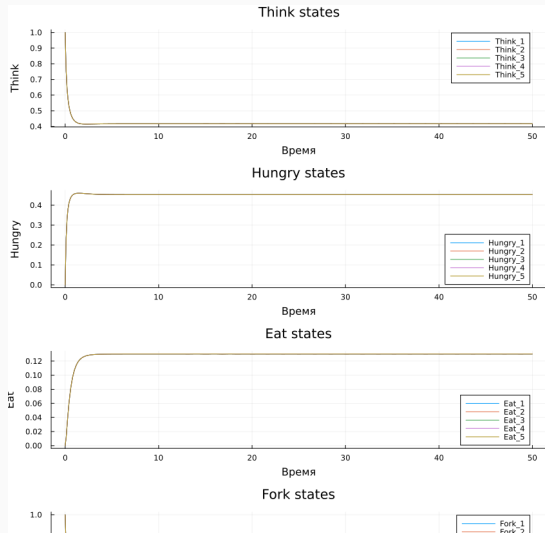
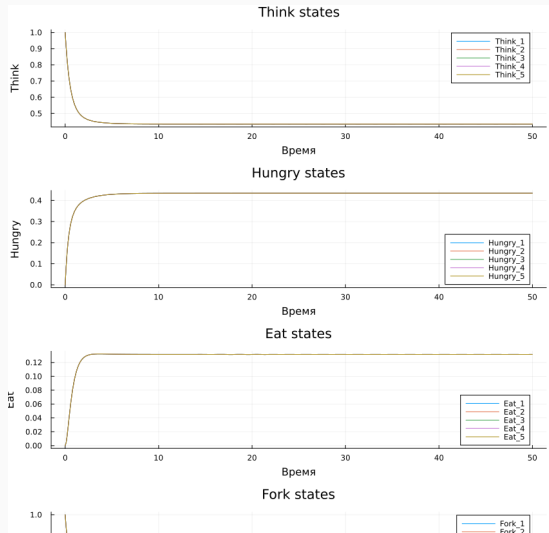
=== Классическая сеть ===
Deadlock обнаружен: true
Последнее время: 6.764

=== Сеть с арбитром ===
Deadlock обнаружен: false
Последнее время: 49.265

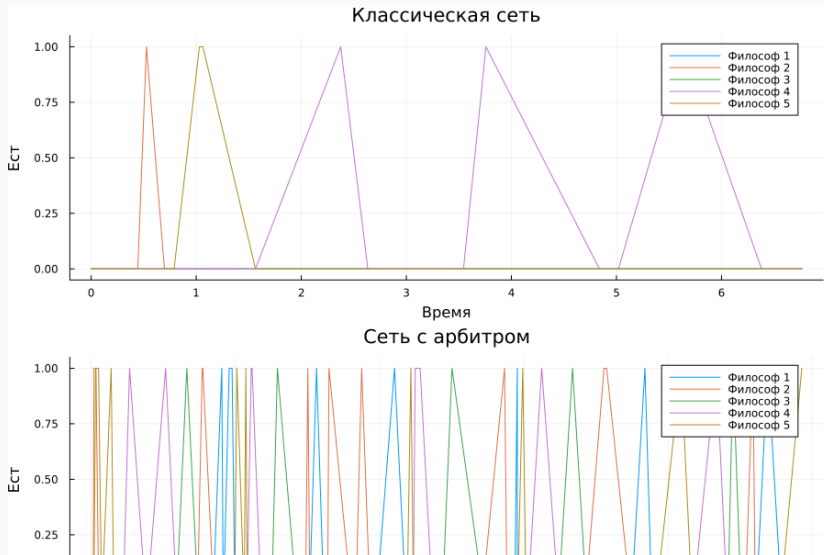
Сохранено:
data/dining_classic.csv
data/dining_arbiter.csv
plots/classic_simulation.png
plots/arbiter_simulation.png
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 4: Результат стохастического эксперимента

Детерминированное моделирование



Сравнение двух вариантов сети



Параметрическое исследование

Проверены:

- $N = 3$;
- $N = 5$;
- $N = 7$;
- несколько различных seed.

Для каждой комбинации рассчитывалось наличие deadlock.

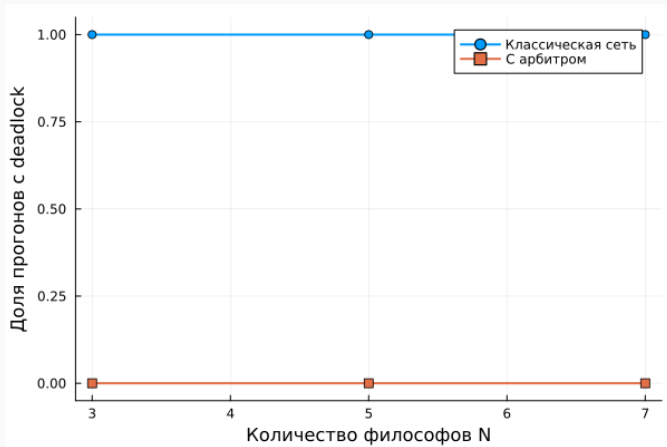


Рисунок 8: Частота возникновения deadlock

Почему используются несколько seed

Алгоритм Гиллеспи является стохастическим.

Поэтому один запуск не даёт полной картины.

Повторные эксперименты позволяют рассчитать долю запусков с взаимной блокировкой и оценить устойчивость результата.

Литературное программирование

Из литературного исходника автоматически создаются:

- Julia-код;
- Jupyter Notebook;
- Quarto-документация.

Таким образом код и его описание хранятся в одном источнике.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ find markdown -type f -name "*.qmd" | sort
markdown/dining_philosophers/dining_philosophers.qmd
markdown/dining_philosophers_animation/dining_philosophers_animation.qmd
markdown/dining_philosophers_deterministic/dining_philosophers_deterministic.qmd
markdown/dining_philosophers_report/dining_philosophers_report.qmd
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ find notebooks -type f -name "*.ipynb" | sort
notebooks/dining_philosophers/dining_philosophers.ipynb
notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
notebooks/dining_philosophers_deterministic/dining_philosophers_deterministic.ipynb
notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$
```

Рисунок 9: Сгенерированные форматы

Проверка воспроизводимости

Все созданные Jupyter Notebook были отдельно выполнены. Это проверяет возможность повторно получить результаты без ручного изменения notebook.

```
Starting kernel event loops.
[NbConvertApp] Writing 6862 bytes to notebooks/dining_philosophers_deterministic/dining_philosophers_deterministic.ipynb
GOTOB0: notebooks/dining_philosophers_deterministic/dining_philosophers_deterministic.ipynb

=====
Выполняется: notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
=====
[NbConvertApp] Converting notebook notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 5945 bytes to notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
GOTOB0: notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb

=====
Выполняется: notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
=====
[NbConvertApp] Converting notebook notebooks/dining_philosophers_report/dining_philosophers_report.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 433891 bytes to notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
GOTOB0: notebooks/dining_philosophers_report/dining_philosophers_report.ipynb

=====
Выполняется: notebooks/dining_philosophers__param/dining_philosophers__param.ipynb
=====
[NbConvertApp] Converting notebook notebooks/dining_philosophers__param/dining_philosophers__param.ipynb to notebook
Starting kernel event loops.
```

Структура проекта

В проекте сохранены:

- модуль сети Петри;
- литературные программы;
- Julia-файлы;
- Jupyter Notebook;
- Quarto-документация;
- CSV;
- PNG;
- GIF.

```
y-simulation-modeling/labs/lab05/project$ find src scripts markdown notebook
s data plots -maxdepth 2 -type f | sort
data/deadlock_parameters.csv
data/deadlock_parameters_summary.csv
data/dining_arbiter.csv
data/dining_arbiter_ode.csv
data/dining_classic.csv
data/dining_classic_ode.csv
markdown/dining_philosophers/dining_philosophers.qmd
markdown/dining_philosophers__param/dining_philosophers__param.qmd
markdown/dining_philosophers_animation/dining_philosophers_animation.qmd
markdown/dining_philosophers_deterministic/dining_philosophers_deterministic
.qmd
markdown/dining_philosophers_report/dining_philosophers_report.qmd
notebooks/dining_philosophers/dining_philosophers.ipynb
notebooks/dining_philosophers__param/dining_philosophers__param.ipynb
notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
notebooks/dining_philosophers_deterministic/dining_philosophers_deterministi
c.ipynb
notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
plots/arbiter_deterministic.png
plots/arbiter_simulation.png
plots/classic_deterministic.png
plots/classic_simulation.png
plots/deadlock_frequency_params.png
plots/final_report.png
plots/philosophers_simulation.gif
scripts/dining_philosophers.jl
scripts/dining_philosophers/dining_philosophers.jl
scripts/dining_philosophers__param.jl
scripts/dining_philosophers__param/dining_philosophers__param.jl
scripts/dining_philosophers_animation.jl
scripts/dining_philosophers_animation/dining_philosophers_animation.jl
scripts/dining_philosophers_deterministic.jl
scripts/dining_philosophers_deterministic/dining_philosophers_deterministic.
jl
scripts/dining_philosophers_report.jl
```

Выводы

- Сети Петри позволяют моделировать конкуренцию процессов за ограниченные ресурсы.
- Классическая задача обедающих философов допускает взаимную блокировку.
- Добавление арбитра меняет правило доступа к ресурсам и предотвращает общий тупик.
- Алгоритм Гиллеспи показывает стохастическую динамику сети.
- Детерминированная модель даёт непрерывное представление тех же процессов.
- Повторные эксперименты позволяют исследовать устойчивость результатов.
- Литературное программирование обеспечивает воспроизводимость вычислений.